



Examensarbete inom Medicinsk Teknik
Grundnivå 15 hp

Framtagning av plattform för utveckling av medicintekniska produkter

En studie om att skapa en plattform för att underlätta
produktutvecklingen av medicintekniska produkter

Thrainn Leo Thorisson och Hugo Persson

Creation of a platform for medical device development

A study to create a platform to facilitate product development of medical devices

Thrainn Leo Thorisson and Hugo Persson

Examensarbete inom Medicinsk Teknik
Grundnivå, 15 hp
Handledare på KTH: Olof Forsberg
Examinator: Dmitry Grishenkov

Kungliga Tekniska Högskolan,
KTH 141 52 Huddinge, Sverige

Sammanfattning

Inom medicinteknisk produktutveckling används regulatoriska krav och internationella standarder för att säkerställa att slutprodukten är säker och håller hög kvalitet. Produktutvecklingen genomgår ofta en lång och noggrann valideringsprocess för att säkerställa att applikationerna uppnår de framställda kraven. Denna process blir väldigt tids- och resurskrävande för utvecklaren. Det finns därför ett stort behov av utvecklingsmetoder för att underlätta den medicintekniska produktutvecklingen.

Examensarbetets syfte är att utveckla och utvärdera en plattform för att underlätta medicintekniska produktutvecklingen genom att automatisera delar av utvecklingsprocessen. Plattformen är avsedd som en grund till medicintekniska applikationer för att underlätta den resurskrävande valideringsprocessen. Den består av ett händelsestyrt ramverk QP/C kombinerat med det grafiska modelleringsverktyget QM som exekveras på en RISC-V-baserad mikrokontroller.

Arbetet innehåller en fungerande simulering av en medicinteknisk applikation som ett praktiskt bevis och ett argument för plattformens egenskaper och underlättande funktioner. QM användes för att grafiskt designa hierarkiska tillståndsmaskiner och automatiskt generera funktionell källkod.

Arbetet har visat att en fungerande plattform, som kan underlätta utveckling av medicintekniska produkter, har skapats. Genom den kompletterande dokumentationen av funktionella krav och metoder bidrar QP/C med ett effektivt standardiseringsarbete. Användandet av aktiva objekt eliminerade behovet av delade resurser och framhävde en alternativ lösning för resurshantering. Studien har visat att en plattform med automatiserade processer har framställts, och att den kan användas i säkerhetskritiska områden som medicinteknisk produktutveckling.

Nyckelord

Aktiva Objekt
Objektorienterad programmering
Medicinteknisk produktutveckling
Medicintekniska produkter
Realtids Händelsedrivet Ramverk
Hierarkiska tillståndsmaskiner
Quantum Leaps
RISC-V
Standarder
Säkerhetskritisk

Abstract

In medical device development, regulatory requirements and international standards are utilized to ensure that the end product is safe and of high quality. The development process often undergoes a long and rigorous validation procedure to guarantee that the application meets the established requirements. This process is highly time-consuming and resource-intensive for the developer. Thus, there is a significant need for development methods that can facilitate medical device development.

The purpose of this thesis is to develop and evaluate a platform designed to simplify medical device development by automating parts of the development process. The platform is intended to serve as a foundation for medical device applications, thereby easing the resource-intensive validation process. It consists of the event-driven framework QP/C combined with the graphical modeling tool QM, executing on a RISC-V-based microcontroller.

The work includes a functional simulation of a medical device application as a practical proof of concept and as an argument for the platform's characteristics and facilitating features. QM was used to graphically design hierarchical state machines and automatically generate functional source code.

The study demonstrates that a functional platform, capable of facilitating the development of medical devices, has been established. Through its complementary documentation of functional requirements and methods, QP/C contributes to efficient standardization work. Furthermore, the utilization of active objects eliminated the need for shared resources, highlighting an alternative solution for resource management. In conclusion, the study shows that a platform with automated processes has been developed and can successfully be applied within safety-critical areas, such as medical device development.

Keywords

Active Object
Object Oriented Programming
Medical device development
Medical products
Real Time Event Framework
Hierarchical State Machines
Quantum Leaps
RISC-V
Standards
Safety-critical

Akronymer

RTEF - Real Time Event Framework
RTOS - Real Time Operating System
ISA - Instruction Set Architecture
ISR - Interrupt Service Routine
Mtime - Machine Time
Mtimecmp - Machine Time Compare
GPIO - General Purpose Input/Output
CPU - Central Processing Unit
MCU - Microcontroller Unit
RAM - Random Access Memory
ECLIC - Enhanced Core Local Interrupt Controller
LIFO - Last In First Out
API - Application Programming Interface
GUI - Graphical User Interface
BSP - Board Support Package
QP - Quantum Plattform
QM - QP Modelar
ISO - International Organization of Standards
IEC - International Electrotechnical Commission
MDR - Medical Device Regulation
FDA - Food and Drug Administration
PDCA - Plan-Do-Check-Act

Innehållsförteckning

1. Inledning.....	1
1.1 Bakgrund.....	1
1.2 Problem.....	1
1.3 Syfte.....	1
1.4 Frågeställning.....	1
2. Metod.....	2
2.1 Val av verktyg.....	2
2.1.1 QP/C.....	2
2.1.2 QM.....	2
2.1.3 QV.....	3
2.1.4 RISC-V.....	4
2.2 Framtagning och verifiering av plattformen.....	4
2.3 Board Support Package (BSP).....	6
2.4 Portning av QP/C till RISC-V.....	6
3. Resultat.....	8
3.1 Resulterande effekter av plattformen.....	8
3.2 Arbete mot standarder.....	9
4. Diskussion.....	10
4.1 Validering av resultat.....	10
4.2 Vidareutveckling av plattformen.....	11
5. Slutsats.....	12
6. Referenslista.....	13
Bilagor.....	17
A - State of the Art.....	17
B - Board Support Package (BSP) för infusionspump.....	28
C - QM genererad kod av aktiva objektet pumpMgr.....	32

1. Inledning

1.1 Bakgrund

Medicintekniska produkter används för att vårda människor och kan vara allt ifrån ett plåster till en ventilator. Produkterna har därför olika säkerhetsnivåer beroende på risker och användningsområden [1]. Då produkterna används inom vården ställs det stora krav på säker funktion. Exempelvis förväntas en ventilator att kunna förse patienten med jämn andning utan att gå sönder. Med det sagt förväntar sig människan att produkten ska fungera säkert och med hög kvalitet vid användning. Medicintekniska applikationer är därför kritiska i sin funktion, då systemfel kan ha livshotande konsekvenser för patienten. För att kunna garantera patientsäkerheten infördes en lösning i form av regulatoriska krav och internationella standarder. Dessa regelverk fungerar som riktlinjer under produktutvecklingen för att säkerställa produkternas kvalitet, tillförlitlighet och riskhantering [2].

1.2 Problem

Att uppnå regulatoriska krav är en lång process, men väsentlig för att medicintekniska produkter ska lanseras på marknaden. Produktutvecklingen sker därför i många steg där prototyper testas och dokumenteras noggrant för att sedan kunna verifieras att de uppnår de regulatoriska kraven [3][4]. På grund av det blir processen väldigt tids- och resurskrävande för tillverkaren. Det finns därför ett stort behov av utvecklingsmetoder som automatiserar delar av produktutvecklingen utan att kompromissa med produktens säkerhet.

1.3 Syfte

Syftet med examensarbetet är att skapa en plattform som bidrar till en delvis automatiserad utvecklingsprocess. Plattformen kommer kombinera en grafisk modelleringsmiljö med ett händelsestyrt ramverk som exekveras på en mikrokontroller. Genom att sammanföra dessa komponenter ska arbetet undersöka hur en sådan plattform kan sänka tröskeln att möta de strikta regulatoriska kraven i utvecklingsprocessen. Plattformen är avsedd att fungera som en grund för produktutveckling, där ramverkets inbyggda arkitektur och metoder framhäver och adresserar specifika krav för medicintekniska produkter. Denna lösning ska gå att implementeras i medicintekniska produkter för att underlätta den tidskrävande och kostsamma utvecklingsprocessen.

1.4 Frågeställning

Examensarbetet kommer att besvara frågeställningen:

- Kan en plattform skapas för att underlätta den medicintekniska produktutvecklingen?

2. Metod

För att underlätta produktutvecklingens noggranna process att uppnå regulatoriska krav, utformades en metod för att lösa problematiken. Genom att kombinera ett tillförlitligt ramverk med en flexibel hårdvara skapades en plattform för att bygga kvalitativa och säkra applikationer. Med ett ramverk som redan arbetade mot att uppnå säkerhetsstandarder, kunde medicintekniska produkter implementera detta för att lättare uppnå kraven.

Skapandet av plattformen krävde grundläggande kunskaper om ramverkets struktur, interna verktyg och API. Hårdvaruarkitekturen granskades med hjälp av datablad, manualer och tillhörande mjukvarubibliotek. Slutligen analyserades ramverkets standarder och de specifika krav som ställdes på mjukvaruarkitektur för att avgöra om plattformen uppfyllde kraven för en medicinteknisk produkt. Bilaga A visar en litteraturstudie som ger en djupare förståelse för metoden.

2.1 Val av verktyg

2.1.1 QP/C

Studien valde ramverket QP/C på grund av dess naturliga utformning för säkerhetsrelaterade system. QP/C utgjorde ett realtids händelsestyrt ramverk (RTEF) och valdes över traditionella realtidsoperativsystem (RTOS), främst på grund av dess händelsestyrda arkitektur som baserades på aktiva objekt [5].

I traditionella RTOS delade trådar ofta globalt minnesutrymme, vilket kräver komplexa synkroniseringsmetoder som mutexer eller tidsfördröjningar för att förhindra datakorruption. Detta ökade risken för funktionsfel såsom kapplöpningstillstånd (race conditions) och dödlägen (deadlocks) [6]. QP/C eliminerade dessa problem genom att isolera resurser i de aktiva objekten som exekverades var och en efter varandra enligt kör-till-slut-principen (run to completion) genom användandet av den inbyggda kärnan QV.

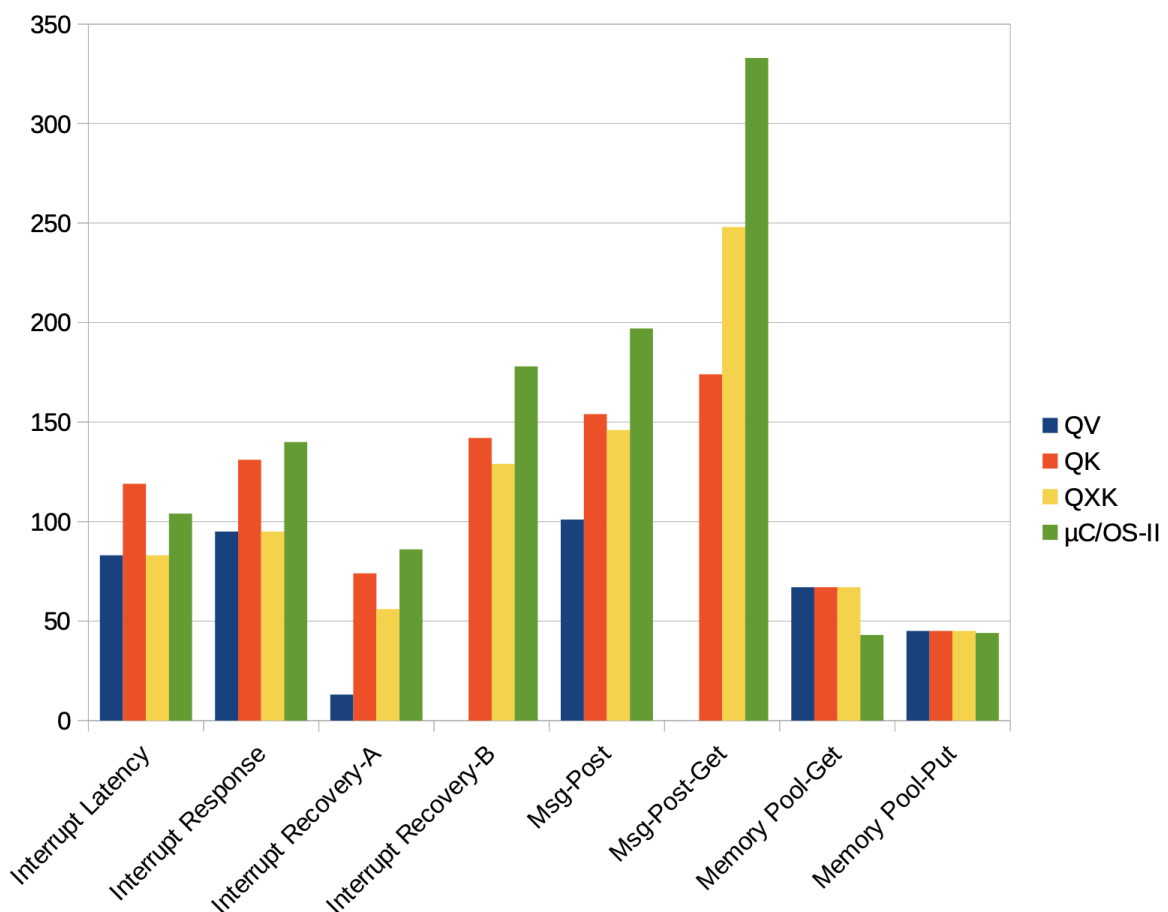
Slutligen valdes basversionen av ramverket QP/C över det mer komplexa och kommersiella ramverket safeQP/C på grund av dess licensfria användning.

2.1.2 QM

För att bygga applikationerna användes modelleringsverktyget QM (QP Modeler) som möjliggjorde en grafisk modellering av hierarkiska tillståndsmaskiner. Programmets aktiva objekt kunde därav definieras med hjälp av tillståndsmaskinerna. Genom att modellera applikationer i QM underlättades framtagningen av systemlogiken samt hantering av tillståndsovergångar. Utöver den underlättade modelleringen valdes QM på grund av dess förmåga att automatiskt generera kod. Den genererade källkoden var MISRA-C kompatibel och kunde enkelt implementeras i projektet genom att använda ramverkets funktioner redan i designen av tillståndsmaskinerna. Därmed visade verktyget en form av tillförlitlighet och underlättade kodutvecklingen av programmet [7].

2.1.3 QV

Valet av QV-kärnan gjordes delvis på grund av dess enkla implementering i ett operativsystems-löst (OS-löst) system vilket plattformen använde. Men den största anledningen för valet var på grund av ett resultat från Quantum Leaps prestandaanalys på QP/C:s inbyggda kärnor. De jämfördes med ett traditionellt RTOS. Prestandaanalysens resultat representeras i figur 1, där låga nummer indikerade bättre resultat [7]. Resultatet demonstrerade QV:s bra resultat jämfört med de andra komponenterna i analysen där de viktigaste resultaten var avbrottsfördröjningen, svarstiden och återhämtningstiden för kärnorna. Testet visade QV kärnans snabba reaktion på avbrott vilket var kritiskt för system som byggde på säkerhet. Detta visar systemets snabba förmåga att sättas i ett säkert tillstånd om fel upptäcktes i systemet. Testerna utfördes på ett liknande billiga kort som projektets RISC-V-baserade mikrokontroller men Quantum Leaps argumenterade för att resultaten representeras väl nog för liknande billiga 32-bitars kort [8].



Figur 1 visar svarsresultat över de olika analyser som utförts på QP/C:s inbyggda kärnor och µC/OS-II, där låga resultat är bra.

2.1.4 RISC-V

Slutligen valdes en mikrokontroller baserad på RISC-V arkitekturen (GD32VF103). Till skillnad från marknadsdominerande alternativ såsom ARM Cortex-M, vilka baseras på stängda arkitekturer, erbjöd RISC-V en helt öppen och licensfri instruktionsuppsättningarkitektur (ISA) [9][10]. Detta medgav den tekniska friheten och flexibiliteten som eftersträvades vid skapandet av plattformen. Valet skapade minimala begränsningar för implementationen av ramverket QP/C.

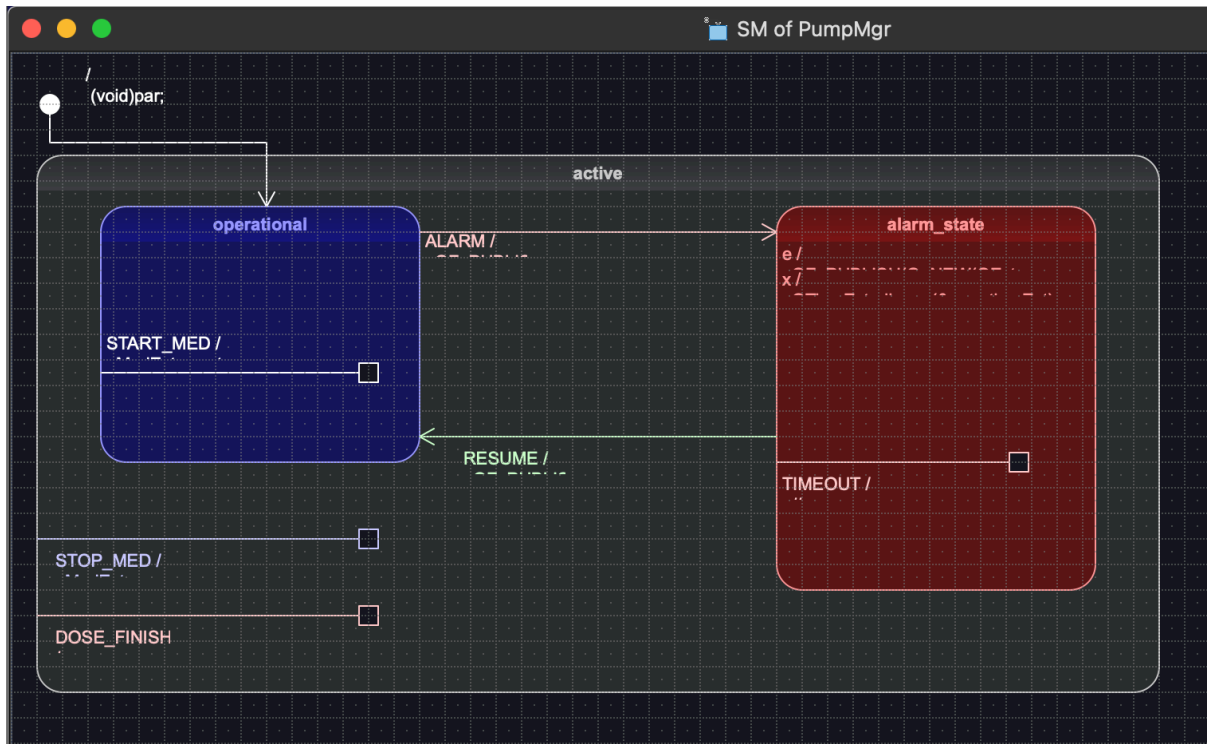
Vidare hade det valda RISC-V-baserade kortet en enkel arkitektur med låg komplexitet. Då QP/C kunde hantera schemaläggningen självständigt genom den inbyggda kärnan QV kunde RISC-V:s OS-lösa arkitektur användas. Strukturen erbjöd en kontrollerad miljö som underlättade portningen av QP/C:s QV-kärna. Hårdvaran valdes således inte utifrån maximal effektivitet, utan för dess kombination av en öppen ISA och låg komplexitet.

2.2 Framtagning och verifiering av plattformen

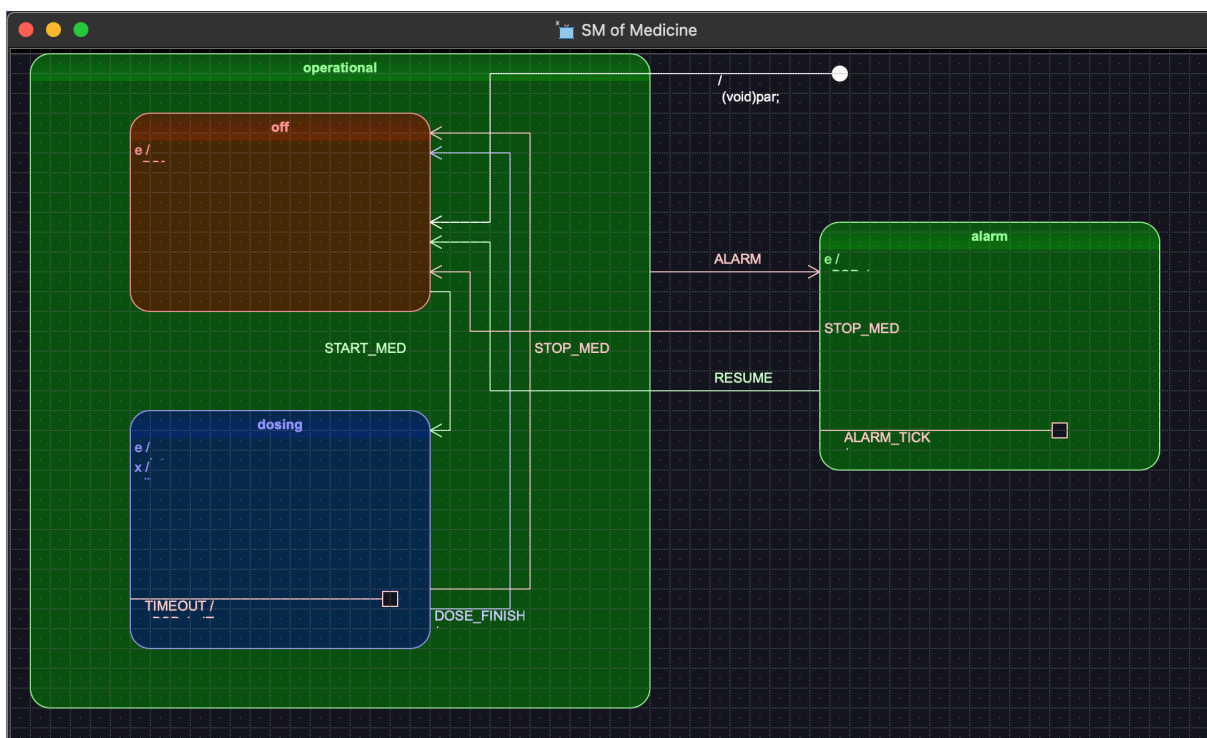
Plattformen utvecklades med hjälp av två program. Första programmet bestod av ett enkelt blinkande exempel vars uppgift var att blinka med en konstant hastighet, bestående av två tillstånd (led on och led off). Programmet var konstruerat av Quantum Leaps men var däremot inte anpassat till ett RISC-V-baserat system. Programmet användes för att enkelt kunna genomföra portningen av verktygen och skapa en bas för plattformen. Denna bas kunde sedan användas till nästa program.

För att redovisa arbetsflödet av plattformen utvecklades nästa program som illustreras i figur 2 och figur 3. Dessa figurer visar de grafiska modellerna för programmets aktiva objekt som skapades i QM. Programmet simulerade en infusionspump med tre olika doseringsnivåer. På det RISC-V-baserade systemet användes tre stycken switchar för att skicka signaler till infusionspumpens kontrollsystem, det aktiva objektet pumpMgr. Vidare användes tre lysdioder för att representera funktionen av de aktiva objekten Medicine. En fjärde switch användes för att aktivera ett larm.

PumpMgr (figur 2) tog emot signaler från kortet och distribuerade dessa vidare till medicinlistans objekt (figur 3). När switch 1, 2 eller 3 aktiverades, skickades en signal (START_MED) med ett specifikt index från PumpMgr till Medicine. Det tillhörande medicinobjektet med matchande index övergick från viloläge till dosering. När larmet aktiverades skickades signalen ALARM_SIG till alla aktörer i systemet och stoppade alla pågående doseringar genom signal prenumeration. Alarmet simulerade att ett systemfel hade detekterats och att systemet säkert kunde övergå till ett kritiskt läge. När switch 4 avaktiverades återgick medicinerna till doseringsfasen.



Figur 2 visar tillståndsmaskinen för aktiva objektet PumpMgr vilket motsvarade systemets kontrollsystem. Modellen bestod av ett supertillstånd (`active`) med två subtillstånd (`operational` och `alarm state`) för alarm och doserings läge.



Figur 3 visar tillståndsmaskinen för Medicine som motsvarade de olika medicinerna. Medicine bestod av ett supertillstånd (`operational`) bestående av två subtillstånd (`off` och `dosing`) och slutligen ett tillstånd för larm.

2.3 Board Support Package (BSP)

Ett Board Support Package användes för att skapa en kommunikation mellan hårdvaran och mjukvaran. BSP:en var mikrokort specifik men inte mjukvaruspecifikt. Det innebar att mjukvaran i sin helhet inte behövde omformuleras vid ett hårdvarubyte utan att det räcker endast med ett omformulerat BSP för att anpassa hårdvaran till mjukvaran. Bilaga B presenterar den fullständiga källkoden för infusionspumpens BSP med beskrivande kommentarer om hur ramverkets funktioner, hårdvarans drivrutiner och pininitieringar användes.

2.4 Portning av QP/C till RISC-V

Som tidigare nämnts kopplades ramverkets mjukvara med det RISC-V-baserade mikrokontroller genom att utveckla ett BSP. Medan ramverket QP/C automatiskt hanterade den interna strukturen via en färdig konfigurationsfil (`qp_port.h`) användes det en verktygskedja med specifika hårdvarubibliotek för kortets processor.

BSP arbetet inleddes med att strukturera funktionen `BSP_init`. Här aktiverades mikrokontrollers systemklockor för de relevanta GPIO-portarna (General Purpose Input/Output). Kortets lysdioder användes som digitala utgångar för att ge en visuell konfirmation att mjukvaran exekverades på kortet.

För att ge ramverket en tidsbas konfigurerades processorns inbyggda maskintimer (`mtime`) och genomfördes med hjälp av registren `mtime` och `mtimecmp`. I ramverkets funktion `QF_onStartup` aktiverades timerns avbrottsflagga i processorns avbrottshanterare, ECLIC (Enhanced Core Local Interrupt Controller). En synkronisering mellan ramverkets och hårdvarans tidshantering krävdes för att utföra portningen. Genom att beräkna ett periodiskt systemtick med hjälp av `BSP_TICKS_PER_SEC` och `TIMER_FREQ`, kunde antalet klockcykler per tidssteg (`tick_step`) bestämmas. `TIMER_FREQ` var definierad enligt ekvation 2.1 och `BSP_TICK_PER_SEC` definierades till 100 Hz:

Ekvation 2.1 visar den definierade klockfrekvensen i verktygskedjans hårdvarubibliotek.

$$TIMER_FREQ = \frac{108\,000\,000\,Hz}{4} = 27\,000\,000\,Hz$$

Ekvation 2.2 visar beräkningen av tidssteget utifrån hårdvarans timerfrekvens:

$$tick_step = \frac{TIMER_FREQ}{BSP_TICKS_PER_SEC} = \frac{27\,000\,000\,Hz}{100} = 270\,000\,Hz$$

Det beräknade tidssteget integrerades i avbrottshanteraren `eclic_mtip_handler` och var avsedd till att exekveras med en frekvens på 100 Hz. Eftersom timern baserades på en kontinuerligt räknande 64-bitars klocka, lästes processorns inbyggda hårdvaruregister `mtime` av vid varje avbrott. Slutligen uppdaterades jämförelsregistret `mtimecmp` med summan av det avlästa `mtime`-värdet och det beräknade tidssteget enligt följande logik i ekvation 2.3:

Ekvation 2.3 visar hur det periodiska avbrottsintervallet beräknades, denna operation schemalade nästa hårdvaruavbrott:

$$mtimecmp = mtime + systemtick$$

Slutligen inkluderades ramverkets periodiska tidshanteringsfunktion (QTIMEEVT_TICK_X) i slutet av avbrottshanteraren för att driva QP:s interna tidshändelser. Följande källkod i figur 4 (tagen från bilaga B) visar konfigurationen av avbrottsintervallet.

```
uint32_t tick_step = TIMER_FREQ / BSP_TICKS_PER_SEC;  
uint64_t next_tick = *(uint64_t volatile *) (TIMER_CTRL_ADDR + TIMER_MTIME);  
*(uint64_t volatile *) (TIMER_CTRL_ADDR + TIMER_MTIMECMP) = next_tick + tick_step;
```

Figur 4 visar källkod för schemaläggning av avbrott i projektets BSP.

3. Resultat

3.1 Resulterande effekter av plattformen

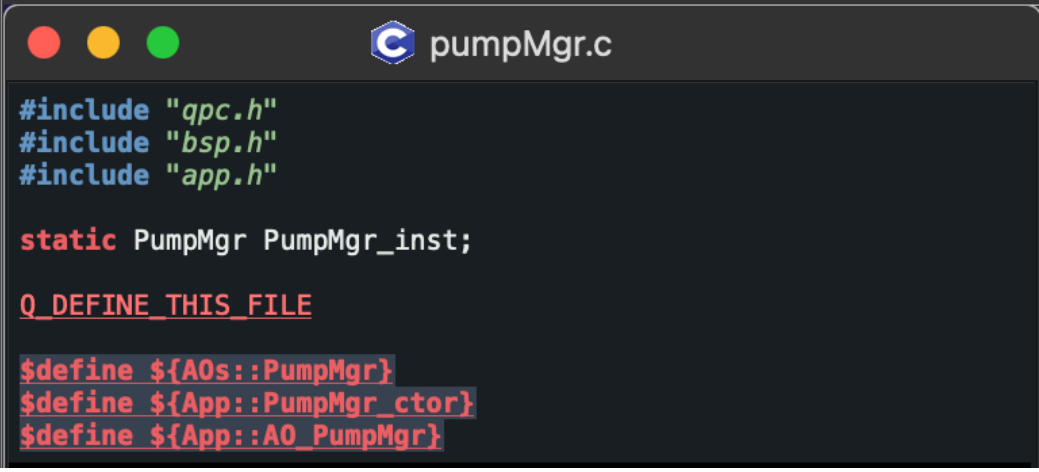
Experimentet visade att portningen av ramverket QP/C till RISC-V, i kombination med modelleringsverktyget QM, resulterade i en fungerande plattform. Ramverkets struktur säkerställde att programmet arbetade mot att uppnå kritiska säkerhetsstandarder och krav som ställs på medicintekniska produkter. Med QV-kärnans händelsedrivna schemaläggning och aktiva objektens resurser isolerade på en tråd ersattes behovet av att använda blockeringsmetoder i form av tidsfördröjningar eller strikta tidsramar vid exekveringen. Det i sin tur eliminerar fel kopplade till delade resurser mellan trådar, såsom kapplöpningstillstånd och dödlägen.

Genom att använda QM kunde hierarkiska tillståndsmaskiner grafiskt modularas. Med nestade tillstånd kunde modellerna minimera upprepning och förenkla tillståndsmaskinens design. Filer som innehöll nyckeldefinitioner för tillståndsmaskinerna skapades i QM, och den fil som krävdes för att generera källkoden för det aktiva objektet pumpMgr visas i figur 6. Filer krävdes för att ge QM funktionen att generera källkod för tillståndsmaskinernas logik, i bilaga C visas den genererade koden för det aktiva objektet pumpMgr vars tillståndsmaskin kan ses i figur 2. Källkoden var MISRA-C-kompatibel och tog en bråkdel av en sekund att framställas, detta visas i figur 5.

QP/C implementerades i QM och bidrog till att tillståndsmaskinerna bestod av ramverkets funktionalitet. Med QM var det möjligt att återgå till den grafiska modellen, redigera den och generera en ny kod på ett effektivt sätt om redigeringar krävdes. På grund av samspelet mellan QM och QP/C krävdes det endast en RISC-V-anpassad BSP (som visas i bilaga B) för att färdigställa plattformen. Denna lösning minskade den manuella programmeringen och risken för mänskliga fel då källkoden genererades.

```
INFO> Model saved: /Users/hugopersson/qm/model.qm
INFO> Code generation started (12:21:10 pm)
INFO> Entire model: /Users/hugopersson/qm/model.qm
INFO> ${:::blink.c} (re)generated file: /Users/hugopersson/qm/blink.c
INFO> Code generation ended (time elapsed 0.004s)
INFO> 1 file(s) generated, 1 file(s) processed, 0 error(s) and 0 warning(s)
```

Figur 5 visar information i terminalen på resultatet av den genererade koden för programmet 'blink' i QM.



```

#include "qpc.h"
#include "bsp.h"
#include "app.h"

static PumpMgr PumpMgr_inst;

Q_DEFINE_THIS_FILE

$define ${AOs::PumpMgr}
$define ${App::PumpMgr_ctor}
$define ${App::AO_PumpMgr}

```

Figur 6 visar en fil i QM med den kod som krävs för att generera kod i C och definiera aktiva objektet pumpMgr.

3.2 Arbete mot standarder

De genomförda experimenten analyserades med fokus på kravspecifikationer samt de standarder som ramverket adresserade. Resultatet visade att användning av aktiva objekt möjliggjorde en strikt mjukvarupartitionering (software partitioning) genom inkapsling av specifika ansvarsområden. Detta minskade systematiska fel genom att isolera eventuella felkällor i enskilda tillstånd. Denna arkitektur stöttade därmed kravet på en modulär mjukvarustruktur och Freedom From Interference som beskrivs i standarderna IEC 62304 respektive ISO 26262 [11][12].

Implementationen av larm redovisade ramverkets publicera/prenumerera-modell, vilket möjliggjorde en effektiv och säker distribution av kritiska signaler. Varje medicin prenumererade på larmsignaler och garanterade att samtliga aktiva enheter reagerade samtidigt vid en incident. När en sådan larmsignal publicerades visade systemet ett förutsägbart och säkert systemtillstånd genom att stanna omedelbart. Genom att använda funktionen Qerror i BSP:n implementerades felhantering tidigt i programmet. Ramverket kunde då säkert sätta systemet i viloläge vid initialiseringfel. Detta deterministiska beteende visade arbetet mot kraven på funktionell säkerhet IEC 61508 [13].

4. Diskussion

4.1 Validering av resultat

Resultatet visar att en fullt fungerande plattform har skapats genom att kombinera RISC-V, ramverket QP/C och modelleringsverktyget QM. Med plattformens praktiska funktionalitet kan resultatet diskuteras genom att undersöka komponenternas egenskaper. Vidare kan denna diskussion avgöra om plattformen kan användas för att underlätta produktutvecklingen.

Ramverket QP/C är certifierat för standarder enligt Software Elements out of Context (SEooC). Det betyder att ramverket i sig inte kan garantera att slutprodukten uppfyller de medicintekniska standarder som QP/C uppnår [22]. Däremot bygger plattformens standardiseringsprocess på QP/C:s dokumentation för att underlätta att arbetet uppnår standarderna. Med hjälp av denna dokumentation får utvecklaren de metoder och tekniker som krävs för att slutprodukten ska uppnå de standarder som ställs på produkten. Detta klargör att med ett certifierat ramverk för bland annat standarderna IEC 61508, IEC 62304 och ISO 26262 får den medicintekniska produkten en god grund för att uppnå funktionella säkerhetsstandarder. Ramverket kan även bidra med att upprätthålla dessa standarder för slutprodukten med hjälp av dokumentationen [22].

Resultatet visar att plattformen kan opereras på ett händelsedrivet ramverk. Det väcker en diskussion om vilken metod som ska föredras när det handlar om schemaläggning av resurser och vilket system som bidrar till högsta möjliga säkerhet. QP/C:s händelsestyrda struktur RTEF bidrar till en väldigt hög grad av inkapsling med hjälp av en kombination av en objektorienterad struktur och användandet av aktiva objekt. Ramverkets inbyggda QV kärna gör att dessa aktiva objekt körs i en gemensam tråd efter varandra då QV hanterar schemaläggningen av händelser genom kör-till-slut-principen [17]. Detta betyder att behovet av komplex trådsynkronisering som mutexar tas bort, då en händelse aldrig kan bli avbruten mitt i exekvering av en annan händelse på samma exekveringskontext. Inkapslingen leder till isolering av resurserna mellan objekten i tråden, vilket bidrar till eliminering av resurshanteringsproblem som kapplöpningstillstånd. I traditionella RTOS sker resurshanteringen genom deterministisk schemaläggning och prioriteringsbaserad exekvering av uppgifter [15]. Eftersom trådar i ett RTOS delar samma globala minnesutrymme, behöver systemet förlita sig på metoder såsom blockering, mutexer eller tidsfördröjning för att schemaläggning och resurshantering av trådar ska fungera korrekt [6]. Det innebär att komplexiteten ökar och att fel såsom kapplöpningstillstånd eller dödlägen kan inträffa, vilket i sin tur kan leda till oförutsägbart beteende och datakorruption. Med RTEF:s inkapsling och uppdelning av resurser skapas det en tillit att den data som ska användas verkligen är korrekt. Ett RTOS erbjuder mer flexibilitet i systemet, men det går inte att säkerställa att resurshanteringsfel som kapplöpningstillstånd elimineras. Det går såklart att använda RTOS för ett säkert system men det kräver mer manuellt arbete för att uppnå detta. Därför kan RTEF och QV uppnå resurssäkerheten på ett mindre komplext sätt då det eliminerar resurshanteringsfelen.

Quantum Leaps hävdar att skapandet av hierarkiska tillståndsmaskiner i QM är en modern arbetsmetod för att utveckla program för säkerhetskritiska områden, bland annat medicinteknik [25]. Med nestade tillstånd kan repetitiva tillståndsövergångar undvikas och skapa en enklare modell, vilket minimerar upprepningar i den grafiska designen. Det i sin tur bidrar till effektivare underhåll av modellen och gör den mer läsbar. Det gör QM till ett kraftfullt verktyg för att skapa inbyggda system. Resultatet visar hur verktygets kodgenerering bidrar till en underlättad kodutveckling. Att istället behöva redigera en komplex kod kan ändringar göras i QM:s lättlästa modell. QM i sin tur genererar en ny, fullständig kod, med de tillagda ändringarna. Enligt Quantum Leaps är den genererade koden så pass optimerad, säker och pålitlig att den direkt kan implementeras i en slutprodukt som är redo för produktion [25]. Det innebär att både tid och arbete kan minimeras för utvecklaren, förutsatt att användandet av verktygen sker på rätt sätt. Med en välstrukturerad designad modell kan produkten uppfylla sin funktion genom kodgenerering. Arbetet visade att med en färdig modul krävdes bara ett hårdvarugränssnitt i form av en BSP för att köra programmet på mikrokortet. Det betyder att ett hårdvarubyte skulle bara kräva en omformulerad BSP för att få programmet att fungera på en annan hårdvara.

4.2 Vidareutveckling av plattformen

Examensarbetet har avgränsats genom att använda det licensfria ramverket QP/C för att undersöka om det kan underlätta medicintekniska produktutvecklingen, men Quantum Leaps hävdar att alla QP ramverk är naturliga val för säkerhetskritiska system [22]. För att vidareutveckla arbetet skulle en övergång till safeQP/C vara det logiska steget. Med safeQP/C:s certifieringssystem kommer ramverket med färdiga artefakter för att programmet ska uppfylla säkerhetscertifiering. Med hjälp av certifieringssystemet ökar sparandet av tid, kostnad och resurser under produktutvecklingen [22]. Trots att safeQP/C är ett kommersiellt verktyg kan det argumenteras att ett sådant ramverk kan vara en bra investering. Med en säkerhetscertifierad mjukvara kan slutprodukten lättare uppnå säkerhetskraven vilket kan leda till mindre kostnad i längden. Quantum Leaps säger att övergången till safeQP/C inte kräver någon modifiering av redan existerande kod som använder QP/C då ramverken delar API specifikationer [23]. Vilket tyder på att ett enkelt byte av ramverk är möjligt om behovet finns. Att införa spårningsprotokollet QP/Spy är också ett förbättringsalternativ för plattformen. Med alternativet att kunna spåra programmet i realtid kan verifieringsprocessen förbättras. Realtidsövervakning effektiviserar därför felsökningen under utvecklingen av produkten och bidrar till en förbättrad dokumentation.

5. Slutsats

Examensarbetet undersökte om en plattform kunde skapas för att underlätta produktutvecklingen för medicintekniska produkter. Studien visar att en kombination av RISC-V, QP/C och QM bidrar till en effektiv, modern och säker plattform som kan användas för utveckling av säkerhetskritiska produkter.

Resultatet bekräftade att en modellbaserad design i QM inte bara effektiviserade utvecklingsprocessen, utan även minskade det manuella arbetet genom automatiserad kodgenerering. Genom att implementera ramverkets funktioner direkt i designen av tillståndsmaskinerna var den genererade kodlogiken direkt kopplad till QP/C. Ramverket i sig visade att med hjälp av dess struktur och dokumentation kunde säkerhetskritiska produkter, som bland annat medicintekniska produkter, lättare uppnå regulatoriska krav såsom IEC 61508, IEC 62304 och ISO 26262.

Vidare hade QV-kärnan en stor roll för plattformens funktionella säkerhet jämfört med traditionella RTOS. Det visar en alternativ lösning av resurshantering för inbyggda system. Genom att använda den öppna och licensfria RISC-V arkitekturen möjliggörs en teknisk frihet för arbetet, vilket bidrar till en smidig process för portning av ramverket. Slutligen med hjälp av plattformen och ett anpassat BSP kan applikationer av olika komplexitet skapas. Vid behov av ett hårdvarubyte krävs bara en omformulerad BSP då plattformens hårdvaruspecifika kod finns där. Detta betyder att resterande mjukvarulogik inte behöver omstruktureras vid användning av en annan hårdvara.

För kommersiellt bruk hade övergången till safeQP/C varit ett logiskt val. Detta på grund av ramverkets mer invecklade säkerhetsprocesser och dokumentation. Implementationen av QP/Spy hade även stärkt plattformen genom att förse en tillämpad felsökningsmetod i realtid för inbyggda system.

Sammanfattningsvis har det konstaterats att plattformen erbjuder en lösning som kan implementeras till medicinteknisk produktutveckling. Detta på grund av plattformens automatisering av högkvalitativ kodgenerering samt användandet av ett väl strukturerat och dokumenterat ramverk för arbetet mot standarder.

6. Referenslista

- [1] Region Gävleborg, “Riskklasser för medicintekniska produkter (MTP)”, Uppdaterad 17 Maj 2022. Citerad 20 Maj 2026.
<https://www.regiongavleborg.se/samverkanswebben/halsa-varld-tandvard/kunskapsstod-och-rutiner/mdr-och-ivdr--nytt-medicintekniskt-regelverk/riskklasser/>
- [2] Daryanani A, U Madekwe. P Baird. & Ehrenfeld J. “Ensuring medical device safety: the role of standards organizations and regulatory bodies” Journal of Medical Systems, 49(1), 16. Uppdaterad 25 Januari 2025, Citerad 22 Maj 2026.
<https://doi.org/10.1007/s10916-025-02150-x>
- [3] Läkemedelsverket, “Utveckling, granskning och “godkännande” av medicintekniska produkter”, Uppdaterad 21 Juni 2023. Citerad 20 Maj 2026.
<https://www.lakemedelsverket.se/sv/medicinteknik/vilka-regler-galler-mig/utveckling-granskning-och-godkannande-av-medicintekniska-produkter>
- [4] Niimi S, “Practice of regulatory Science (Development of medical devices),” YAKUGAKU ZASSHI, vol. 137, no. 4,, doi: 10.1248/yakushi.16-00244-3, Uppdaterad Maj 2017, Citerad 18 Maj 2026. <https://pubmed.ncbi.nlm.nih.gov/28381720/>
- [5] Quantum Leaps, “QP real-time event frameworks (RTEFs),” Modern Embedded Software, Uppdaterad 11 Oktober 2025. Citerad 10 April 2026.
<https://www.state-machine.com/products/qp>
- [6] Hamilton, L, “FreeRTOS mutex a developers guide to race free embedded systems”, Uppdaterad 20 Mars 2026. Citerad Maj 18, 2026. <https://bugprove.com/freertos-mutex/>
- [7] Quantum Leaps, “QM Model-Based Design Tool,” Modern Embedded Software, Uppdaterad 29 September 2025. Citerad 10 April 2026.
<https://www.state-machine.com/products/qm>
- [8] Quantum Leaps, “QP Performance Tests and Results”, Uppdaterad Jul 2016. Citerad 19 Maj 2026. https://www.state-machine.com/doc/AN_QP_Performance.pdf
- [9] Joseph Y, “ARM CORTEX-M3”, Newnes, Uppdaterad 2010. Citerad 10 Maj 2026.
<https://wiki.ifsc.edu.br/mediawiki/images/2/29/MIPM3TUG.pdf>
- [10] RISC-V International. “About RISC-V - RISC-V International, Uppdaterad 15 April 2026. Citerad 10 April 2026. <https://riscv.org/about/>
- [11] ISO, “IEC 62304: Medical device software – Software life cycle processes”. Uppdaterad 2021. Citerad 26 April 2026. <https://www.iso.org/obp/ui/en/#iso:std:iec:62304:ed-1:v1:en>
- [12] Florian F, Stefan L, Sirui, “Automated Freedom from Interference Analysis

for Automotive Software”, Citerad 22 Maj 2026.

<https://kops.uni-konstanz.de/server/api/core/bitstreams/d37af1b1-0222-44d0-ab3d-e4d3729dea92/content>

[13] MESCO Engineering GmbH, “Functional Safety according to IEC 61508”, Uppdaterad 24 Mars 2026. Citerad 22 Maj 2026.

<https://mesco-engineering.com/en/services/functional-safety/>

[14] Kartik G, “Concurrency”, Citerad 23 Maj 2026.

https://oscourse.github.io/slides/concurrency_race_and_locks.pdf

[15] Kowalski R, Huy F: Real-Time Operating Systems (RTOS) 101 | NASA, Uppdaterad 2016. Citerad 10 April 2026.

https://www.nasa.gov/wp-content/uploads/2016/10/482489main_4100_-_rtos_101.pdf

[16] Wind River Wind River, “Intro to Real-Time Operating Systems (RTOS)” , Citerad 22 Maj 2026. Citerad 10 April 2026 <https://www.windriver.com/solutions/learning/rtos>

[17] Quantum Leaps, “Key concept: Active Object”, Modern Embedded Software, Uppdaterad 7 Januari 2023. Citerad 12 April 2026.

<https://www.state-machine.com/active-object>

[18] Kaashoek.F, Saltzer.J: Principles Of Computer Systems Designs | Science Direct, Uppdaterad 2009. Citerad 26 Mars 2026.

<https://www.sciencedirect.com/topics/computer-science/operating-system-kernel>

[19] Quantum Leaps, “non-preemptive QV Kernel”, Citerad 26 Mars 2026.

https://www.state-machine.com/qpc/arm-cm_qv.html

[20] Quantum Leaps, “Preemptive Non-Blocking QK Kernel”, Citerad 26 Mars 2026.

https://www.state-machine.com/qpc/arm-cm_qk.html

[21] Quantum Leaps, “Preemptive ‘Dual-Mode’ QXK kernel”, Citerad 28 Mars 2026.

https://www.state-machine.com/qpc/arm-cm_qxk.html

[22] Quantum Leaps, “Overview”, Citerad 28 Mars 2026.

<https://www.state-machine.com/qpc/>

[23] Quantum Leaps, “Functional Safety Viewpoint”, Citerad 23 Maj 2026,

https://www.state-machine.com/qpc/sds-qp_fusa.html

[24] Jama Software, “Understanding IEC 62061 - JAMA Software”, Uppdaterad 30 Januari 2025. Citerad 24 April 2026

<https://www.jamasoftware.com/requirements-management-guide/industrial-manufacturing-development/iec-62061-functional-safety-for-machinery-systems/?kw=&cpn=18653661776&ut>

[m_source=google&utm_medium=paid_search&utm_campaignname=emea-search-dsa-nonb-max-value&utm_term=&utm_content=%7Badid%7D&campaign_id=18653661776&adgroup_id=139928452942&adgroup=dsa-dsa-all-pages&ad_id=%7Badid%7D&creative=614759725145&device=c&network=g&matchtype=&placement=&referrer=%7Breferrerurl%7D&gad_source=1&gad_campaignid=18653661776&gbraid=0AAAAADL9ZkLKmVQfzAVcOp2nMn2f58AQY&gclid=CjwKCAjwqubPBhBOEiwAzgZX2tpdkCYgZq4Ge7fDmQYBoFsOvuQSIT1H5w9hP6eM9Rt8rlqlaOKOjhoCIRgQAvD_BwE](https://www.google.com/adsense/adsense?utm_source=google&utm_medium=paid_search&utm_campaignname=emea-search-dsa-nonb-max-value&utm_term=&utm_content=%7Badid%7D&campaign_id=18653661776&adgroup_id=139928452942&adgroup=dsa-dsa-all-pages&ad_id=%7Badid%7D&creative=614759725145&device=c&network=g&matchtype=&placement=&referrer=%7Breferrerurl%7D&gad_source=1&gad_campaignid=18653661776&gbraid=0AAAAADL9ZkLKmVQfzAVcOp2nMn2f58AQY&gclid=CjwKCAjwqubPBhBOEiwAzgZX2tpdkCYgZq4Ge7fDmQYBoFsOvuQSIT1H5w9hP6eM9Rt8rlqlaOKOjhoCIRgQAvD_BwE)

[25] Quantum Leaps, “State Machines”, Citerad 4 April 2026.

<https://www.state-machine.com/qm/sm.html>

[26] Quantum Leaps, “About QM”, Citerad 4 April 2026.

https://www.state-machine.com/qm/index.html#ab_what

[27] Quantum Leaps, “QTools: QUTest Trace-Based testing”, Citerad 4 April 2026.

https://www.state-machine.com/qtools/qutest.html#qutest_about

[28] Quantum Leaps, “About QSPY”, Citerad 4 April 2026.

https://www.state-machine.com/qtools/qspy.html#qspy_about

[29] Quantum Leaps, “QTools: QP/Spy Software Tracing”, Citerad 4 April 2026.

<https://www.state-machine.com/qtools/qpspy.html>

[30] Asanović K. RISC-V International, “RISC-V ANNUAL REPORT 2025”,

Uppdaterad 2025. Citerad 2 April 2026.

<https://riscv.org/wp-content/uploads/2026/01/RISC-V-Annual-Report-2025.pdf#:~:text=It%20projects%20that%20our%20market,adoption%20in%20the%20right%20places>

[31] Herdt V, Große D, Drechsler R, “Enhanced Virtual Prototyping”, Springer Cham.

DOI:10.1007/978-3-030-54828-5, ISBN 978-3-030-54827-8. Uppdaterad 2021. Citerad 1 Mars 2026.

[32] Patterson D, Hennessy J: Computer Organization and Design: The Hardware/Software Interface, RISC-V Edition. Uppdaterad 2017. Citerad 31 Mars 2026

https://www.cs.sfu.ca/~ashriram/Courses/CS295/assets/books/HandP_RISCV.pdf

[33] GeeksforGeeks, “Instruction set architecture and microarchitecture,” GeeksforGeeks, Uppdaterad 25 Oktober 2025. Citerad 28 Februari 2026.

<https://www.geeksforgeeks.org/computer-organization-architecture/microarchitecture-and-instruction-set-architecture/>

[34] Cajander A, “Inbyggda system med RISC-V”, upplaga 1:2. Lund: Studentlitteratur,

Uppdaterad 2023. Citerad 31 Mars 2026.

- [35] Nuclei Spec, “8. TIMER Unit Introduction documentation.”, Uppdaterad 3 Jan 2023. Citerad 30 Mars 2026. https://doc.nucleisys.com/nuclei_spec/isa/timer.html#timer-overview
- [36] Läkemedelsverket, “Regelverk för medicintekniska produkter”, Uppdaterad 16 September 2025. Citerad 30 Mars 2026. <https://www.lakemedelsverket.se/sv/medicinteknik/regelverk#hmainbody5>
- [37] Läkemedelsverket, “CE-märkning”, Uppdaterad Januari 10 2021. Citerad 30 Mars 2026. <https://www.lakemedelsverket.se/sv/medicinteknik/tillverka/vagen-till-ce-market>
- [38] Svenska institutet för standarder, SIS, “Vad är en standard?”, Citerad 30 Mars 2026. <https://www.sis.se/standarder/vad-ar-en-standard/>
- [39] ISO - Standards | ISO, Citerad 30 Mars 2026. <https://www.iso.org/standards.html>
- [40] SEK svensk elstandard, “IEC International Electrotechnical Commission”, Citerad 30 Mars 2026. <https://elstandard.se/iec/>
- [41] Kiwa, “ISO 13485 - Certifiering av ledningssystem för medicintekniska produkter”, Uppdaterad 9 December 2022. Citerad 22 April 2026. <https://www.kiwa.com/se/sv/tjanster/certifiering/iso-13485-medicintekniska-produkter/>
- [42] ISO, “ISO 13485:2016 Quality management for medical devices”, Uppdaterad Januari 2016. Citerad 22 April 2026, <https://www.iso.org/files/live/sites/isoorg/files/store/en/PUB100377.pdf>
- [43] Coursera, “PDCA: The Plan-Do-Check-Act Cycle”, Uppdaterad 23 Oktober 2024. Citerad 20 April 2026. <https://www.coursera.org/articles/pdca>

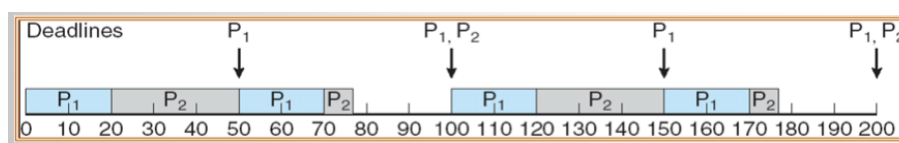
Bilagor

A - State of the Art

A.1 RTOS

Ett realtidsoperativsystem (RTOS) är ett operativsystem som med förebyggande syfte använder sig av multitasking för realtidsapplikationer. Syftet med operativsystemet är att hantera hårdvara och mjukvara på ett sätt som garanterar att uppgifter slutförs inom en strikt tidsram. För att synkronisering av schemalaggningen ska möjliggöras använder RTOS metoder som bland annat implementationer av mutexer för att se till att en uppgift körs i taget. Mutexer ser då till att en tråd får tillgång till resurser genom att blockera resterande trådar [6]. Problem kan uppstå om synkroniseringen för olika tråders tidsramar är ur fas. Det kallas kapplöpningstillstånd (race conditions) och beskriver hur två eller flera trådar som jobbar med samma variabler kan leda till oförutsägbara värden [14].

RTOS tekniska egenskaper såsom determinism innebär att operativsystemets tjänster tar en känd eller förväntad mängd tid på sig att utföras. För varje givet tillstånd ska det komma en unik utdata för att sedan kunna bestämma nästa tillstånd. RTOS schemaläggning fungerar genom fördröjningsfunktioner där uppgifter tilldelas prioriteter (Priority Based Preemptive Scheduling), på så sätt bestäms när en uppgift ska köras. En högprioriterad uppgift kan omedelbart avbryta en lägre prioriterad uppgift och få tillgång till den globala datan [15]. Det i sin tur kan leda till fel i form av dödlägen, vilket är när två eller flera uppgifter blockerar varandra på grund av att händelser har samma prioriteringar. Systemet blockerar omedvetet dessa trådar vilket leder till att ingen uppgift körs [14]. Slutligen strävar ett RTOS efter att exekvera alla högprioriterade uppgifter först och ser till att uppgifter exekveras snabbt inom strikta tidsramar [15]. Figur 7 illustrerar hur förebyggande prioritetsbaserad schemaläggning fungerar.



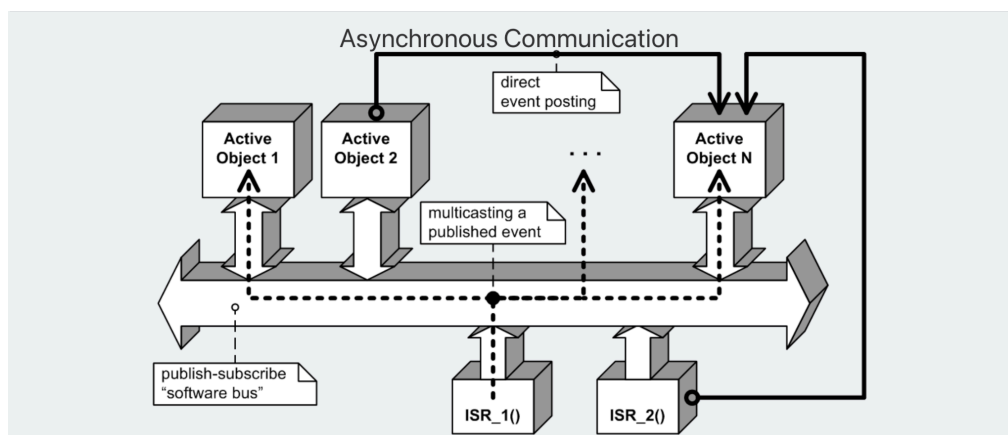
Figur 7 ger en visuell beskrivning av hur prioriteringen sker i ett RTOS, där en uppgift med högre prioritet exekveras före de lågprioriterade uppgifterna. Figuren är tagen från NASA, Real Time Operating Systems (RTOS) 101.

RTOS är ett väldigt populärt verktyg som förknippas med inbyggda system. Med RTOS determinism och förutsägbarhet kan uppgifter exekveras på väldigt korta tider. Många industrier utnyttjar dessa egenskaper och implementerar RTOS till deras produkter. Exempelvis används RTOS på medicintekniska produkter såsom MRI (Magnetkamera), ventilatorer och kirurgiutrustning [16].

A.2 Active Objects

Aktiva objekt kan användas i ett händelsestyrt system och genomför en asynkron kommunikation genom utbyte av meddelanden som framstår som händelser. Objektet bidrar med ökad inkapsling av data vilket betyder att delar av systemet har begränsad åtkomst till trådarnas attribut. Istället för att ha global åtkomst mellan trådar delas resurserna upp i systemet, vilket minskar risken för fel i koden. Vidare genom att använda händelser mellan aktiva objekt utförs uppgifter i systemet utan att blockera trådar [17].

Aktiva objekt tilldelas en egen tråd vars livslängd beror på en händelsekö. När en händelse inträffar, aktiveras det aktiva objektet och genomför uppgiften enligt kör-till-slut-principen (run to completion). När händelsekön sedan är tom hamnar aktiva objektet i viloläge och sparar därför minne i det inbyggda systemet. Flera aktiva objekt kan sedan kommunicera med varandra med hjälp av händelser vilket visas i figur 8. Som tidigare nämnt ska aktiva objektens trådar följa principen RTC, vilket garanterar att uppgifter genomförs innan nästa uppgift påbörjar sin exekvering. Detta för att förhindra fel såsom dödlägen och kapplöpningstillstånd [17].



Figur 8 visar designmönstret för olika aktiva objekt i ett system. Händelser levereras asynkront mellan aktiva objekt som sedan kan exekvera uppgiften som exempelvis avbrott.

A.3 Kärnor

I ett operativsystem används kärnor som en komponent för att skapa ett abstrakt gränssnitt och möjliggör för mjukvaran att läsa eller skriva instruktioner till hårdvaran. Kärnans främsta uppgift är att hantera systemets resurser genom schemaläggning. Schemaläggningen avgör vilka processer eller trådar som ska få tillgång till processorns CPU och vid vilken tidpunkt. Såsom i ett RTOS möjliggör det multitasking i systemet [18].

QP/C-ramverket kan implementeras till ett operativsystem, olika processorarkitekturer och kompilatorer på grund av dess anpassningsbarhet. Ramverket tillhandahåller olika kärnor som definieras till olika miljöer. För att implementera ramverket till processorer som av ett operativsystemlös (OS-lös) hårdvara används inbyggda kärnor. Val av inbyggd kärna beror på systemets krav på realtidsegenskaper och resurstillgång. Nedan visas en tabell på QP/C-ramverkets olika kärnor som används för OS-lösa processorer.

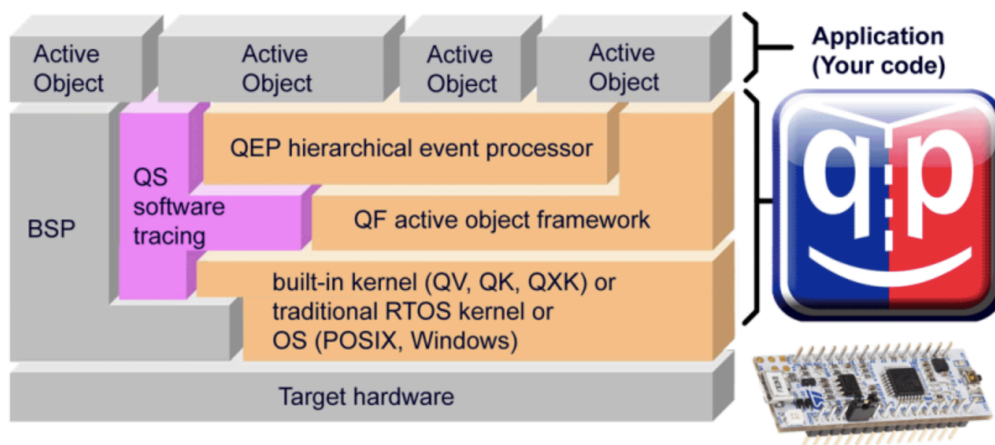
Tabell 2.1: Inbyggda kärnor skapade av Quantum leaps för schemaläggning

QV	En händelsestyrd kärna exekverar ett aktivt objekt i taget med en schemaläggning som bygger på prioritering för olika händelser. Bearbetning av händelser sker genom kör-till-slut-principen (run to completion) [19].
QK	En icke-blockerande kärna använder en gemensam stack och tillåter händelser med hög prioritet att avbryta lågprioriterade händelser. Detta görs genom att ge den högprioriterade händelsen få åtkomst till processorn före den lågprioriterade händelsen [20].
QX	Kärnan kombinerar den traditionella blockeringen av trådar hos ett RTOS med QK-kärnan. QX används för att blanda händelsestyrda aktiva objekt med en blockerande RTOS kod [21].

A.4 Quantum Leaps

Quantum Leaps är ett företag som tillhandahåller ett så kallat Real Time Event Framework (RTEF) som kan användas i inbyggda system. Företaget använder aktiva objekt och hierarkiska tillståndsmaskiner (HSM) för att strukturera det händelsedrivna ramverket som ett alternativt substitut för traditionella RTOS. Quantum Leaps har tagit fram ramverket QP/C och den kompletterande versionen safeQP/C samt verktygen QP modeller (QM) och QP/Spy [5].

Figur 9 visar strukturen som Quantum Leaps använder för inbäddad programvara.



Figur 9 visar Quantum Leaps struktur och komponenter för systemet. Figuren är tagen från Quantum Leaps, Real-Time-Event Frameworks (RTEFs).

A.4.1 QP/C RTEF

Ramverket QP/C gör ett intryck i dagens industri och används i produkter inom medicin, IoT (Internet of Things), robotik, transport och mera. Ramverket är byggt på asynkrona och händelsedrivna aktiva objekt samt används i realtidsbaserade inbyggda system. Ramverket har en objektorienterad struktur där resurser inkapslas med hjälp av aktiva objekt och hög abstraktionsnivå med hjälp av hierarkiska tillståndsmaskiner [22]. QP/C:s mjukvarustruktur är certifierat för att uppnå funktionell säkerhet och har dokumentation som kan användas för att slutprodukten ska uppnå standarder såsom IEC 62304, ISO 26262, IEC 61508 och MISRA-C [23].

QP/C kan användas utan ett operativsystem där den traditionella RTOS funktionaliteten ersätts med hjälp av så kallade inbyggda kärnor. Denna metod effektiviserar minnet hos hårdvarans processor på grund av den händelsedrivna arkitekturen som används för schemaläggning av uppgifter. Detta på grund av att aktiva objekt sätts i viloläge när inga händelser uppstår. Om ramverket kombineras med ett operativsystem kan så kallade tredjeparts kärnor användas. Denna metod används bara om en befintlig mjukvara kräver blockeringsmetoder men visar anpassningsbarheten på ramverket [22].

Slutligen har ramverket en komplementerande ramverksversion (safeQP/C) som lägger extra vikt på programvarusäkerhetlivscykel (SSL) [22]. Versionerna bygger på metoder för att bedöma risker och fastställer hur säkerhetskrav ska användas för att uppnå funktionell säkerhet [24]. SafeQP/C är en kommersiell utökning av det licensfria ramverket QP/C som har säkerhet i fokus. Vid skapandet av safeQP/C utfördes faroanalys och riskbedömning (HARA) på det befintliga ramverket QP/C. Under analysen identifierades svagheter inom den funktionella modellen samt ramverkets API (Application Programming Interface). Säkerhetslösningar implementeras successivt till safeQP/C och verifierades för att säkerställa att svagheter eliminerades. SafeQP-certifieringspaketet förser utvecklare med färdiga artefakter vilket möjliggör tidsbesparing, minskning av risker och sänkning av kostnader av applikationscertifieringar [23].

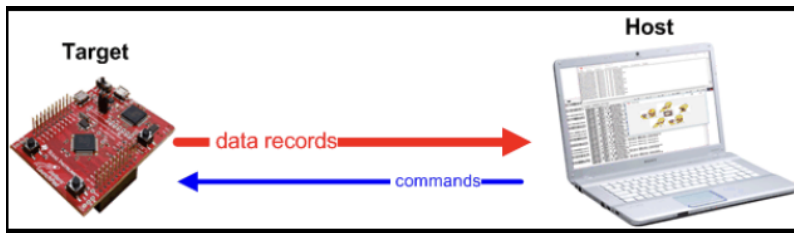
A.4.2 QM

QM är ett modellbaserat verktyg som används för att grafiskt designa tillståndsmaskiner. Den modellerade tillståndsmaskinens egenskaper kan sedan genereras till kod. Tekniken används för att effektivisera kodutvecklingen av händelsestyrda system och kan implementeras till inbyggda system. QM är designat för att skapa hierarkiska tillståndsmaskiner som bygger på användandet av arv, nästade tillstånd, abstraktion och polymorfism. Hierarkiska tillståndsmaskiner minimerar repetition och ökar återanvändning av moduler. Det innebär att beteendet definieras för ärvande tillstånd (substrates) från föräldrarnas tillstånd (superstate). Metoden beskrivs då som ett objektorienterat verktyg som består av klasser där tillståndsmaskiner skapas och associerar med varandra [25].

Med den grafiska designen av hierarkiska tillståndsmaskiner skapar QM en möjlighet att dela upp mjukvaran i aktiva objekt för att separera logiken och göra modellen mer läsbar. QM använder ett användarvänligt gränssnitt, följt av kodgenerering vilket skapar ett användbart verktyg.

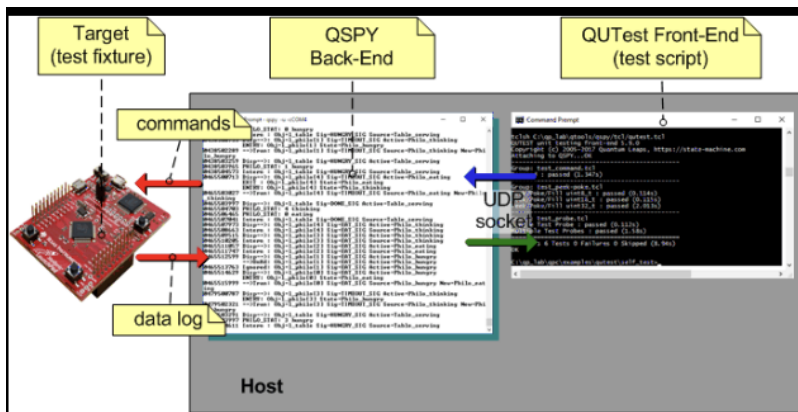
Genereringen är möjlig då QM baseras på QP/C:s definierade ramverksarkitektur vilket i sin tur skapar väldefinierade regler för kodgenereringen. Den automatiska kodgenereringen används till inbyggda system i C eller C++ och återspeglar tillståndsmaskinens design. QM bidrar då till produktionskvalitet i form av spårbarhet genom att eliminera mänskliga felet vid kodutvecklingen och förenklar felsökning av den grafiska designen [26].

Med den grafiska beskrivningen på hierarkiska tillståndsmaskiner integreras modellen i projektmappen som header-filer (.h) och implementationsfiler (.c). Det skapar en kontrollerad och flexibel källkodsstruktur. Konceptet minskar behovet att handskriva källkoden eller kombinera med befintlig tredjepartskod. Slutligen är verktygets struktur anpassat till användaren och ger full kontroll över den grafiska designen [26]. Figur 10 visar verktygets användargränssnitt och en visuell beskrivning av utvecklingsmiljön .



Figur 11 visar kommunikation mellan mikrokontrollern och datorn med hjälp av spårningssystemet QP/Spy. Figuren är tagen från Quantum Leaps, QP/Spy Software Tracing.

QUTest använder metoden kod under testning (CUT) som injicerar kontrollerad indata till systemet. Verktöget validerar att de genererade exekveringsspåren exakt korresponderar med en fördefinierad sekvens av förväntade händelser. Detta möjliggör en förutbestämd testmiljö för både testdriven utveckling och beteendedriven utveckling. Utöver enhetstester användas QUTest för integrationstestning direkt på hårdvarens system och verifierar mjukvarans interaktion med den fysiska miljön. Figur 12 visar arbetsflödet mellan QP/Spy, QUTest och hårdvaran [29].



Figur 12 visar kommunikationen mellan hårdvaran (target), Q/Spy och QUTest. Figuren är tagen från Quantum Leaps, About QUTest.

A.5 RISC-V

RISC-V är en öppen licensfri Instruction Set Architecture (ISA) som från början var skapad i ett akademiskt syfte men har växt inom industri. Företag har inte bara tillägnat RISC-V:s öppna struktur utan aktivt börjat bygga produkter runt den [30]. Hårdvaran är designad för att vara enkel och anpassningsbar vilket möjliggör enkla komponenttillägg. Hårdvaran kan då byggas till ett mer komplext system och fortfarande uppfylla de kommersiella kraven.

RISC-V är en meningsfull plattform för inbyggda system. På grund av dess enkla ISA struktur där allting sitter i samma integrerade krets som kallas MCU (Microcontroller Unit). Funktionaliteten är utspridd i olika kretsar som sedan sammankopplas av olika bussar. I grund och botten finns det alltid en processor (CPU) som kopplas till ett programminne, dataminne eller I/O-enheter [31].

A.5.1 ISA - Instruction Set Architecture

ISA är ett abstrakt gränssnitt mellan hårdvara och hårdvarunära programvara som omfattar informationen för att skapa program i maskinspråk på ett korrekt sätt. Kommunikationen som ISA utgör till processorn (CPU) är av operationer såsom att addera data, ladda data eller hoppa till en annan uppgift i minnet [32]. Till skillnad från mer läsbara programmeringsspråk är ISA begränsad av antal nummer av platser som är inbyggda i hårdvaran [33]. Dessa platser kallas register. En RISC-V-processor innehåller totalt 32 stycken processorregister eller minnesplatser och beroende på processortyp på kortet kan varje processorregister innehålla 32, 64 eller 128 bitar. I vilket nummer ett register är tilldelat benämns som x0 till x31, där x0 är av en generell typ. Utöver de 32 registren finns det en så kallad programräknare (pc) som alltid pekar ut nästa instruktion som ska genomföras. Med det sagt har dessa register en load/store - arkitektur som används för att adressera, flytta eller hantera data mellan processorns register och RAM-minnet [34].

A.5.2 Minneshantering och stack arkitektur i RISC-V

Ett inbyggt system fördelar verktygskedjan programmets statiska datasektioner i början av mikrokontrollens RAM-minne. Utöver denna statiska allokering finns det behov att reservera minne dynamiskt under exekveringen av programmet. Detta sker i en del av minnet som kallas för heap, vilket expanderar linjärt mot högre minnesadresser i processorns minne [34].

En annan datastruktur som kallas för stack ligger i den övre delen av RAM-minnet (högsta tillgängliga adressen). Till skillnad från hepen växer stacken nedåt mot lägre adresser. En stack är en linjär datastruktur och fungerar enligt principen last in, first out (LIFO). Detta innebär att ett objekt som senast lades in till stacken är det första som kommer ut. Stacken fungerar som ett korttidsminne för processorn och används framförallt för att ha ordning på funktionsanrop. När programmet hoppar till en funktion sparas adressen för den tidiga positionen så att programmet kan återvända när funktionen är genomförd. Denna adress kallas för en stackpekare. Vidare har stacken ordning på lokala variabler som bara är användbara under tiden en specifik funktion genomförs. De lokala variablerna tas sedan bort från stacken när funktionen har avslutats. Metoden blir väldigt användbar vid användning av

operativsystem men även inbyggda som använder OS-lösa system där RAM-minnet är begränsat [34].

A.5.3 Timers

Inbyggda system ska kunna hantera tidsstyrda uppgifter på ett tillförlitligt sätt. Då ett system ska kunna göra flera händelser om och om igen krävs det en kontroll över mikrokontrollers avbrotts klockor. Vanligtvis är en MCU utrustad med flera oberoende klockor som kan drivas av antingen interna klocksignaler eller externa källor. För en RISC-V finns det sju räknare med varierande komplexitet [34].

Ett exempel på en sådan räknare är RISC-V processorns processorkärna vid namnet Nuclei som är utrustad med ECLIC (Enhanced Core Local Interrupt Controller) för att hantera avbrottsignaler. RISC-V:s CPU innehåller en realtidsräknare vid namnet mtime (machine timer) vilket är ett 64-bitars register som räknar uppåt med en konstant hastighet. Vidare finns ytterligare ett 64-bitars register vid namnet mtimecmp (machine time compare). Detta register fungerar som ett jämförelseregister. RISC-V:s CPU jämför dessa register ständigt och kontrollerar ett enda villkor som är att om mtime är lika med eller större än mtimecmp ($\text{mtime} \geq \text{mtimecmp}$). Om villkoret uppfylls, aktiveras en flagga som sedan plockas upp av RISC-V:s MCU. Processorns CPU plockar i sin tur denna flagga och skickar en avbrottsignal som hanteras av mjukvaran. Mekanismen opererar genom så kallad Memory-Mapped I/O (MMIO). Vilket innebär att klockornas kontrollregister är direkt allokerade till specifika adresser i processorns basadress. Ett förutbestämt värde är satt för jämförelseregistret som sedan kan ändras löpande med hjälp av mjukvaran för att uppdatera resterande adresser i minnet och generera flera avbrottssignaler i systemet [35]. Metoden blir därav användbar när händelser ska upprepas i ett system.

A.6 Regelverk för medicintekniska produkter

För att medicintekniska produkter på marknaden ska bidra till samhället och främja innovation finns det regelverk på såväl nationell nivå som internationell nivå. Europa reglerar medicintekniska produkter huvudsakligen av förordningen Medical Device Regulation (MDR) [36]. Internationellt ställer även det amerikanska regelverket U.S. Food and Drug Administration (FDA) krav på produkter som ska användas i USA. Det innebär att produkterna uppfyller de krav och lagar som landet har stiftat. Syftet med regelverk är att ställa tydliga och öppna krav på produkter som ska vara i bruk. Detta säkerställer att produkter följer en förutbestämd lista av krav som sedan säkerställer tillförlitligheten på produkten. Produkten blir då säker för både användare, patienter och miljön. Uppfyller produkten de lagkrav som finns för medicintekniska produkter i Europa märks produkterna med CE märkning. Denna symbol är ett tecken på att produkten som används har uppfyllt de lagkrav och standarder som ställs [37].

A.6.1 Standarder

Standarder används för att säkerställa att det finns enhetliga och transparenta lösningar på återkommande problem. Detta innebär bland annat att produkter och tjänster uppnår högsta möjliga kvalitet samt användarvänlighet för kunder. Genom att följa standarders krav, tillvägagångssätt eller rekommendationer kan produkter skapas för att uppnå säkerhet och kvalitet [38][39]. Organisationer som ISO (International Organization for Standardisation) eller IEC (International Electrotechnical Commission) har standarder som täcker olika områden såsom säkerhet, kvalitet, miljö, elektronik. Dessa organisationer är oberoende, icke-statliga organisationer [40].

A.6.2 ISO 13485

ISO 13485 är en standard för ett kvalitetsledningssystem. Standarden är anpassad för företag som designar, tillverkar, testar, säljer och ger garanti på medicintekniska produkter. Genom att följa denna standard och myndigheternas krav på kvalitet blir företagets produkt certifierat enligt ISO 13485. Standarden utgår från den generella standarden för kvalitetsledningssystem (ISO 9001) men är utökad med specifika krav som anpassas för medicintekniska sektorn. Standarden omfattar hela produktlivscykeln, från design och utveckling till produktion och underhåll [41].

Standarden lägger större vikt på riskhantering och beslutomfattning kring risker för den slutförda produkten. Fokus ligger på risker förknippade med medicintekniska produkter, säkerhet och kvalitet. Riskhanteringen görs med hjälp av ramverket Plan-Do-Check-Act cycle (PDCA). Denna komponent är en iterativ metod som fungerar som en strukturerad process för att säkerställa att krav och säkerhetsmål uppfylls vid varje implementation av produkten [42][43].

A.6.3 IEC 62304:2015

IEC 62304:2015 är en standard skriven av organisationen IEC (International Electrotechnical Commission). Den definierar krav som en mjukvara bör ha i en medicinteknisk produkt. Standarden beskriver processer, aktiviteter och uppgifter som skapar ett gemensamt ramverk för livscykelprocesser förknippade med medicintek. Standarden fungerar mer specifikt för att ha uppehåll på programvaran. Det betyder att om den skulle bli inbäddad eller integrerad så kommer inte standarden täcka den medicintekniska produkten även om den helt baseras på programvaran. Produkten bör täckas av en annan standard [11].

A.6.4 IEC 61508:2010

Standarden IEC 61508:2010 arbetar mot funktionell säkerhet för både hårdvara och mjukvara. Den lägger stor vikt på att arbetet ska riktas mot att minska olyckor och skador vid fel, för att säkerställa maximal säkerhet för människor, maskineri och miljön. IEC 61508:2010 ställer höga krav på utvecklingsmetodik samt implementering av hårdvara/mjukvara. Standarden används som ett stöd när en idé ska förvaltas till en fullständig implementering. Med funktionell säkerhet använder standarden sig av analysmetoder som FMEA/FMEDA (Failure Modes, Effects and Diagnostic Coverage

Analysis) och SIL-klassificeringar (Safety Integrity Level). Utöver detta kan IEC 61508:2010 komplettera relevanta standarder som till exempel ISO 13849 (standard för maskinsäkerhet) och andra IEC standarder [13].

B - Board Support Package (BSP) för infusionspump

```
// RISC-V -----
#include "gd32vf103.h"
#include "gd32vf103_rcu.h"
#include "gd32vf103_gpio.h"
#include "n200_timer.h"

// QP -----
#include "qpc.h"
#include "bsp.h"
#include "qp_port.h"
#include "app.h"

//=====
Q_DEFINE_THIS_FILE // file name for assertions
//=====

/* =====
 * Q_onError
 *
 * Global felhanterare för QP-ramverket. Anropas automatiskt om en assertion
 * (ett kritiskt programfel) fallerar. Funktionen stänger av alla avbrott,
 * tändar LED B0 och B1 för att indikera en krasch, och går sedan in i en
 * permanent loop för att frysa systemet.
 * ===== */
Q_NORETURN Q_onError(char const * const module, int_t const id) {
    QF_INT_DISABLE();
    gpio_bit_set(GPIOB, GPIO_PIN_0 | GPIO_PIN_1);
    for (;;)
}

/* =====
 * eclic_mtip_handler
 *
 * Systemets timer-avbrottshanterare som körs var 10:e millisekund (100 Hz).
 *
 * Central funktionalitet:
 * 1. Beräknar och schemalägger nästa hårdvaruavbrott genom att addera ett
 *    tidssteg (tick_step) till den interna 64-bitars klockan (MTIME) och
 *    skriva till jämförelseregistret (MTIMECMP).
 * 2. Driver QP:s interna tidshändelser via QTIMEEVT_TICK_X().
 * 3. Läser av de fyra switcharna (A8-A5) och använder kantdetektering
 *    (jämförelse med föregående tillstånd) för att upptäcka när en switch
 *    ändrar läge. Detta förhindrar att identiska signaler skickas kontinuerligt.
 * 4. Allokerar och postar relevanta händelser (START/STOP för medicinpumpar,
```

```

*      samt ALARM/RESUME för larm) till Pump-managerns kö.
* ===== */
void eclic_mtip_handler(void) {
    uint32_t tick_step = TIMER_FREQ / BSP_TICKS_PER_SEC;
    uint64_t next_tick = *(uint64_t volatile *) (TIMER_CTRL_ADDR + TIMER_MTIME);
    *(uint64_t volatile *) (TIMER_CTRL_ADDR + TIMER_MTIMECMP) = next_tick +
tick_step;

    QTIMEEVT_TICK_X(0U, (void *)0);

    if (AO_PumpMgr == (QActive *)0 || AO_Medicine[0] == (QActive *)0) {
        return;
    }

    uint8_t nowA8 = gpio_input_bit_get(GPIOA, GPIO_PIN_8);
    uint8_t nowA7 = gpio_input_bit_get(GPIOA, GPIO_PIN_7);
    uint8_t nowA6 = gpio_input_bit_get(GPIOA, GPIO_PIN_6);
    uint8_t nowA5 = gpio_input_bit_get(GPIOA, GPIO_PIN_5);

    static uint8_t lastA8, lastA7, lastA6, lastA5;

    // --- Medicin 0 (Switch A8) ---
    if (nowA8 != lastA8) {
        lastA8 = nowA8;
        MedEvt *mev = Q_NEW(MedEvt, (nowA8 == 1U) ? START_MED_SIG : STOP_MED_SIG);
        mev->medId = 0U;
        QACTIVE_POST(AO_PumpMgr, (QEvt *)mev, 0U);
    }

    // --- Medicin 1 (Switch A7) ---
    if (nowA7 != lastA7) {
        lastA7 = nowA7;
        MedEvt *mev = Q_NEW(MedEvt, (nowA7 == 1U) ? START_MED_SIG : STOP_MED_SIG);
        mev->medId = 1U;
        QACTIVE_POST(AO_PumpMgr, (QEvt *)mev, 0U);
    }

    // --- Medicin 2 (Switch A6) ---
    if (nowA6 != lastA6) {
        lastA6 = nowA6;
        MedEvt *mev = Q_NEW(MedEvt, (nowA6 == 1U) ? START_MED_SIG : STOP_MED_SIG);
        mev->medId = 2U;
        QACTIVE_POST(AO_PumpMgr, (QEvt *)mev, 0U);
    }
}

```

```

// --- Larm (Switch A5) ---
if (nowA5 != lastA5) {
    lastA5 = nowA5;
    QEvt *e = Q_NEW(QEvt, (nowA5 == 1U) ? ALARM_SIG : RESUME_SIG);
    QACTIVE_POST(AO_PumpMgr, e, 0U);
}
}

/* =====
* BSP_init
*
* Initierar mikrokontrollerns hårdvaruperiferi. Aktiverar klockorna för
* GPIO-port A och B, nollställer samt konfigurerar LED-pinnarna (B0-B2) som
* digitala utgångar (Push-Pull), och konfigurerar switch-pinnarna (A8-A5)
* som ingångar med internt pull-up-motstånd (IPU).
* ===== */
void BSP_init(void) {
    rcu_periph_clock_enable(RCU_GPIOA);
    rcu_periph_clock_enable(RCU_GPIOB);

    gpio_bit_reset(GPIOB, GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2);

    gpio_init(GPIOB, GPIO_MODE_OUT_PP, GPIO_OSPEED_50MHZ,
              GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2);

    gpio_init(GPIOA, GPIO_MODE_IPU, GPIO_OSPEED_50MHZ,
              GPIO_PIN_8 | GPIO_PIN_7 | GPIO_PIN_6 | GPIO_PIN_5);
}

/* =====
* LED- / Lampkontrollfunktioner
*
* Abstraktionslager för att styra kretskortets lysdioder (port B).
* - BSP_ledOn/Off: Tänder eller släcker en specifik LED via bit-shifting.
* - BSP_ledAllOff: Släcker alla tre lysdioder samtidigt.
* - BSP_ledToggle: Skiftar tillstånd på en LED genom att invertera biten (XOR).
* ===== */
void BSP_ledOn(uint8_t id) {
    gpio_bit_set(GPIOB, (1U << id));
}

void BSP_ledOff(uint8_t id) {
    gpio_bit_reset(GPIOB, (1U << id));
}

```

```

void BSP_ledAllOff(void) {
    gpio_bit_reset(GPIOB, GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2);
}

void BSP_ledToggle(uint8_t id) {
    GPIO_OCTL(GPIOB) ^= (1U << id);
}

/* =====
 * QF_onStartup
 *
 * Konfigurerar och startar hårdvarutimern precis innan QP-ramverkets
 * händelseloop drar igång. Aktiverar timer-avbrottet i avbrottshanteraren
 * (ECLIC) med lägsta prioritet, beräknar starttiden för det första avbrottet
 * med ett litet extra andrum för CPU:n, och slår slutligen på globala avbrott.
 * ===== */
void QF_onStartup(void) {
    eclic_irq_enable(CLIC_INT_TMR, 1, 1);
    uint32_t tick_step = TIMER_FREQ / BSP_TICKS_PER_SEC;

    uint64_t volatile *mtime      = (uint64_t volatile *) (TIMER_CTRL_ADDR +
TIMER_MTIME);
    uint64_t volatile *mtimecmp = (uint64_t volatile *) (TIMER_CTRL_ADDR +
TIMER_MTIMECMP);

    *mtimecmp = *mtime + (tick_step * 2);

    QF_INT_ENABLE();
}

/* =====
 * QV_onIdle
 *
 * QP-ramverkets idle-loop som körs automatiskt när det inte finns några
 * händelser kvar att hantera i systemet. Ser till att globala avbrott är
 * aktiverade så att systemtimern kan väcka processorn igen.
 * ===== */
void QV_onIdle(void) {
    QF_INT_ENABLE();
}

```


[illegible]


```

}

//${AOS::PumpMgr::SM::active::operational} .....
QState PumpMgr_operational(PumpMgr * const me, QEvt const * const e) {
    QState status_;
    switch (e->sig) {
        //${AOS::PumpMgr::SM::active::operational::START_MED}
        case START_MED_SIG: {
            MedEvt const *pe = (MedEvt const *)e;
            me->medWasRunning[pe->medId] = true;
            MedEvt *newEvt = Q_NEW(MedEvt, START_MED_SIG);
            newEvt->medId = pe->medId;

            QACTIVE_POST(AO_Medicine[pe->medId], (QEvt *)newEvt, me);
            status_ = Q_HANDLED();
            break;
        }
        //${AOS::PumpMgr::SM::active::operational::ALARM}
        case ALARM_SIG: {
            QF_PUBLISH(Q_NEW(QEvt, ALARM_SIG), me);
            status_ = Q_TRAN(&PumpMgr_alarm_state);
            break;
        }
        default: {
            status_ = Q_SUPER(&PumpMgr_active);
            break;
        }
    }
    return status_;
}

//${AOS::PumpMgr::SM::active::alarm_state} .....
QState PumpMgr_alarm_state(PumpMgr * const me, QEvt const * const e) {
    QState status_;
    switch (e->sig) {
        //${AOS::PumpMgr::SM::active::alarm_state}
        case Q_ENTRY_SIG: {
            QF_PUBLISH(Q_NEW(QEvt, ALARM_TICK_SIG), me);

            QF_PUBLISH(Q_NEW(QEvt, ALARM_TICK_SIG), me);
            uint8_t i;
            for (i = 0U; i < N_MEDS; ++i) {
                if (me->medWasRunning[i]) {
                    MedEvt *tickEvt = Q_NEW(MedEvt, ALARM_TICK_SIG);
                    tickEvt->medId = i;
                }
            }
        }
    }
    return status_;
}

```

```

        QACTIVE_POST(AO_Medicine[i], (QEvt *)tickEvt, me);
    }
}

QTimeEvt_armX(&me->timeEvt, BSP_TICKS_PER_SEC/15, BSP_TICKS_PER_SEC/15);
status_ = Q_HANDLED();
break;
}

//${AOs::PumpMgr::SM::active::alarm_state}
case Q_EXIT_SIG: {
    QTimeEvt_disarm(&me->timeEvt);
    status_ = Q_HANDLED();
    break;
}

//${AOs::PumpMgr::SM::active::alarm_state::RESUME}
case RESUME_SIG: {
    QF_PUBLISH(Q_NEW(QEvt, RESUME_SIG), me);
    uint8_t i;
    for (i = 0U; i < N_MEDS; ++i) {
        if (me->medWasRunning[i]) {
            MedEvt *newEvt = Q_NEW(MedEvt, START_MED_SIG);
            newEvt->medId = i;
            QACTIVE_POST(AO_Medicine[i], (QEvt *)newEvt, me);
        }
    }
    status_ = Q_TRAN(&PumpMgr_operational);
    break;
}

//${AOs::PumpMgr::SM::active::alarm_state::TIMEOUT}
case TIMEOUT_SIG: {
    uint8_t i;
    for (i = 0U; i < N_MEDS; ++i) {
        if (me->medWasRunning[i]) {
            MedEvt *tickEvt = Q_NEW(MedEvt, ALARM_TICK_SIG);
            tickEvt->medId = i;
            QACTIVE_POST(AO_Medicine[i], (QEvt *)tickEvt, me);
        }
    }
    status_ = Q_HANDLED();
    break;
}

default: {
    status_ = Q_SUPER(&PumpMgr_active);
    break;
}
}

```

